

# Recurrent Neural Networks

Avi Mohan

# Today's Class

- Main goal: sequence generation
- What are sequences
  - Example: natural language
- Supervised learning to Sequence Modeling
  - Models of word probabilities
  - N-grams, advantages, and disadvantages
- Recurrent Networks
  - LSTMS
  - GRUs

# Sequence Generation

- Task of producing a new sequence of symbols
  - One element at a time (can also have multiple outputs)
  - Usually **conditioned** on previous outputs
  - Example: natural language generation, machine translation, summarization, chatbots, time series forecasting, etc.
- Collection of elements
  - that can be **repeated**
  - whose **order** matters, and
  - that can have **variable (potentially infinite) length**
- Models discussed so far don't do well with sequential data.

# Supervised Learning to Sequence Modeling

|              | Supervised Learning                                                             | Sequence Modeling                                        |
|--------------|---------------------------------------------------------------------------------|----------------------------------------------------------|
| Data         | $\{x_i, y_i\}_{i=1}^{N_{train}}$                                                | $\{x_i\}$                                                |
| Model        | $\hat{y} = f(w; x)$                                                             | $p(x) = f(w; x)$                                         |
| Loss         | $\mathcal{L}(w) = \frac{1}{N_{train}} \sum_{i=1}^{N_{train}} l(y_i, \hat{y}_i)$ | $\mathcal{L}(w) = \sum_{i=1}^{N_{train}} \log f(w; x_i)$ |
| Optimization | $w^* = \operatorname{argmin}_w \mathcal{L}(w)$                                  | $w^* = \operatorname{argmax}_w \mathcal{L}(w)$           |

# Models of $p(\mathbf{x})$

- “Mary had a little lamb.”
- Simplest model: independence

$$p(\mathbf{x}) = \prod_{i=1}^T p(x_i)$$

- $p(\text{"Mary"}) \times p(\text{"had"}) \times p(\text{"a"}) \times p(\text{"little"}) \times p(\text{"lamb."})$
- Large enough corpus gives probabilities
- **Problem:**  $w^*$  ends up producing sentences like  
“*The the the the...*”

| Word   | $p(x_i)$ |
|--------|----------|
| the    | 0.049    |
| be     | 0.028    |
| ...    | ...      |
| really | 0.0005   |
| ...    | ...      |

# Models of $p(\mathbf{x})$

- “Context”: everything so far,  $x_1, x_2, \dots, x_t$

$$p(x_{t+1}) = p(x_{t+1} | \text{context})$$

- Chain rule
- $p(\text{"Mary"}) \times p(\text{"had"} | \text{"Mary"}) \times p(\text{"a"} | \text{"Mary", "had"}) \times \dots$
- **Problem: Not scalable**
  - $N_{\text{english}}$  = number of words
  - $l$  = length of context
  - $N_{\text{english}}^{l+1}$
- Cannot be stored, searched, or even *approximated*.

# Models of $p(x)$

- Fix the Context size: **N-grams**
  - Only condition on the past N words.  
$$p(x_{t+1}) \approx p(x_{t+1} | x_t, x_{t-1}, \dots, x_{t-N+1})$$
  - Example: 2-grams condition only on  $x_t, x_{t-1}$
- **Not a good model:**  $p(\cdot | \text{"a"}, \text{"lamb"})$ 
  - Context can stretch across thousands of words.
- **Data table is still too large:**  $N_{english}^{N+1}$

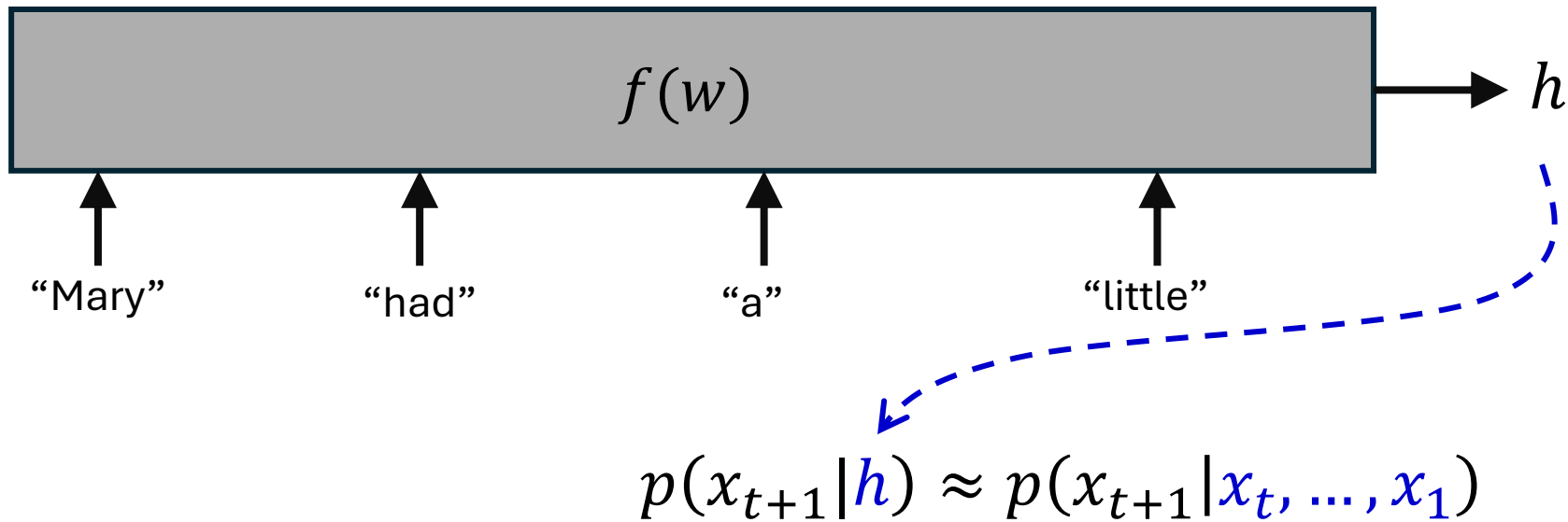
Here at Google Research we have been using word [n-gram models](#) for a variety of R&D projects, such as [statistical machine translation](#), speech recognition, [spelling correction](#), entity detection, information extraction, and others. While such models have usually been estimated from training corpora containing at most a few billion words, we have been harnessing the vast power of Google's datacenters and distributed processing [infrastructure](#) to process larger and larger training corpora. We found that there's no data like more data, and scaled up the size of our data by one order of magnitude, and then another, and then one more - resulting in a training corpus of *one trillion words* from public Web pages.

We believe that the entire research community can benefit from access to such massive amounts of data. It will advance the state of the art, it will focus research in the promising direction of large-scale, data-driven approaches, and it will allow all research groups, no matter how large or small their computing resources, to play together. That's why we decided to share *this enormous dataset with everyone*. We processed 1,024,908,267,229 words of running text and are publishing the counts for all *1,176,470,663 five-word sequences that appear at least 40 times*. There are 13,588,391 unique words, after discarding words that appear less than 200 times.

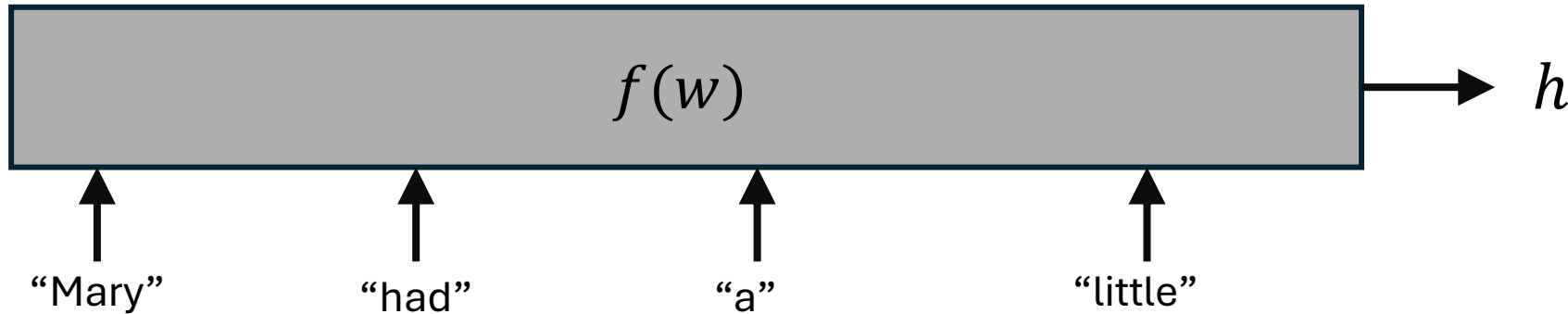
Watch for an announcement at the Linguistics Data Consortium ([LDC](#)), who will be distributing it soon, and then order your set of 6 DVDs. And [let us hear from you](#) - we're excited to hear what you will do with the data, and we're always interested in feedback about this dataset, or other potential datasets that might be useful for the research community.

# Learning $p(\cdot)$ models

- We need to:
  - summarize the current context
  - generate the next word based on this summary.

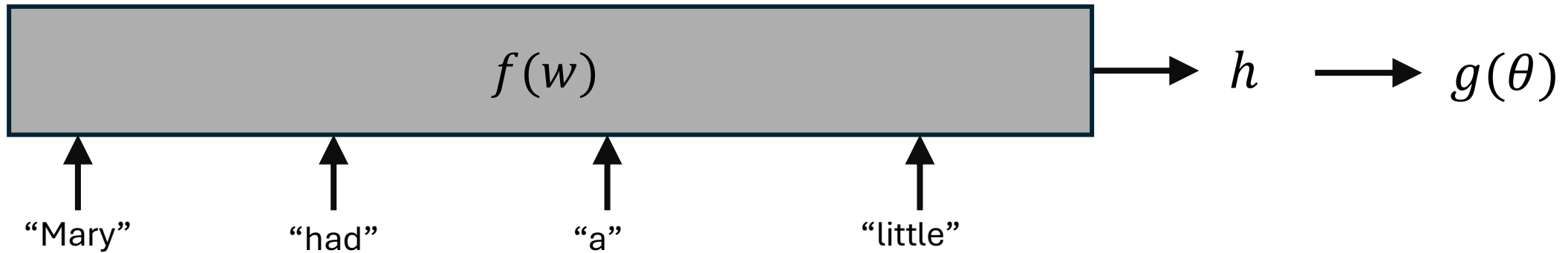


# Learning $p(\cdot)$ models



- Desiderata:
  - network must take **variable-length inputs**
  - $h$  must be **order-sensitive**
  - **learnability** (differentiable: GD)
  - individual changes can have large effects ("not")
  - preserve **long term dependencies**.

# Learning $p(\cdot)$ models



- Desiderata for  $g(\theta)$ :
  - individual changes can have large effects
  - returns a valid **probability distribution**

# Today's Class

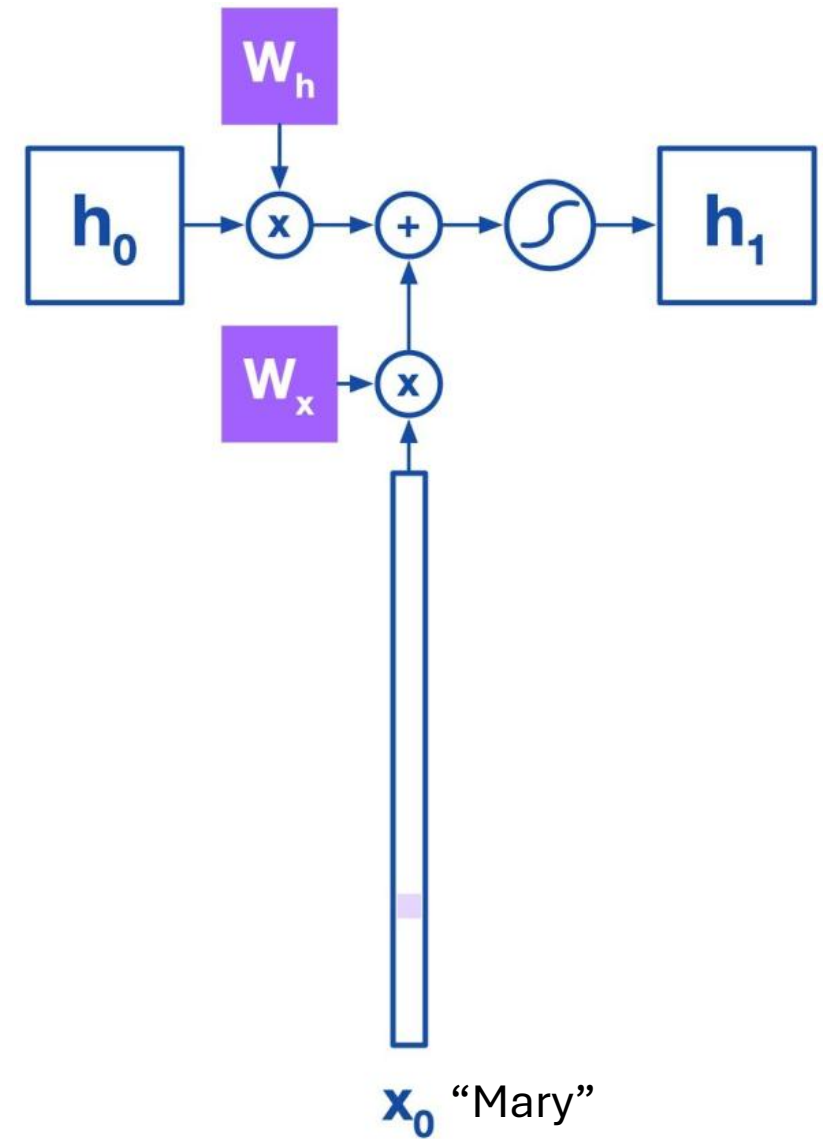
- Main goal: sequence generation
- What are sequences
  - Example: natural language
- Supervised learning to Sequence Modeling
  - Models of word probabilities
  - N-grams, advantages, and disadvantages
- Recurrent Networks
  - LSTMS
  - GRUs

# Recurrent Neural Networks

- The state variable  $h_t$  stores information from the context observed so far

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

- Weighted combination of current input and previous context.



# Recurrent Neural Networks

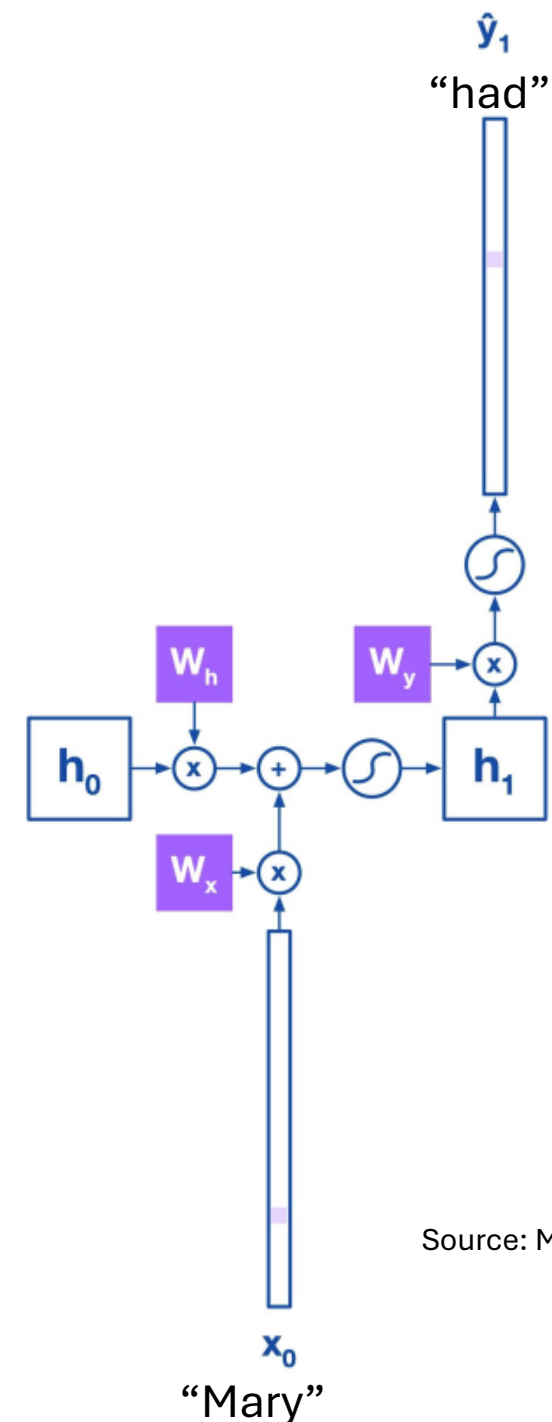
- The state variable  $h_t$  stores information from the context observed so far

$$h_t = \tanh(W_h h_{t-1} + W_x x_t)$$

- Predict the target (next word) from this state vector

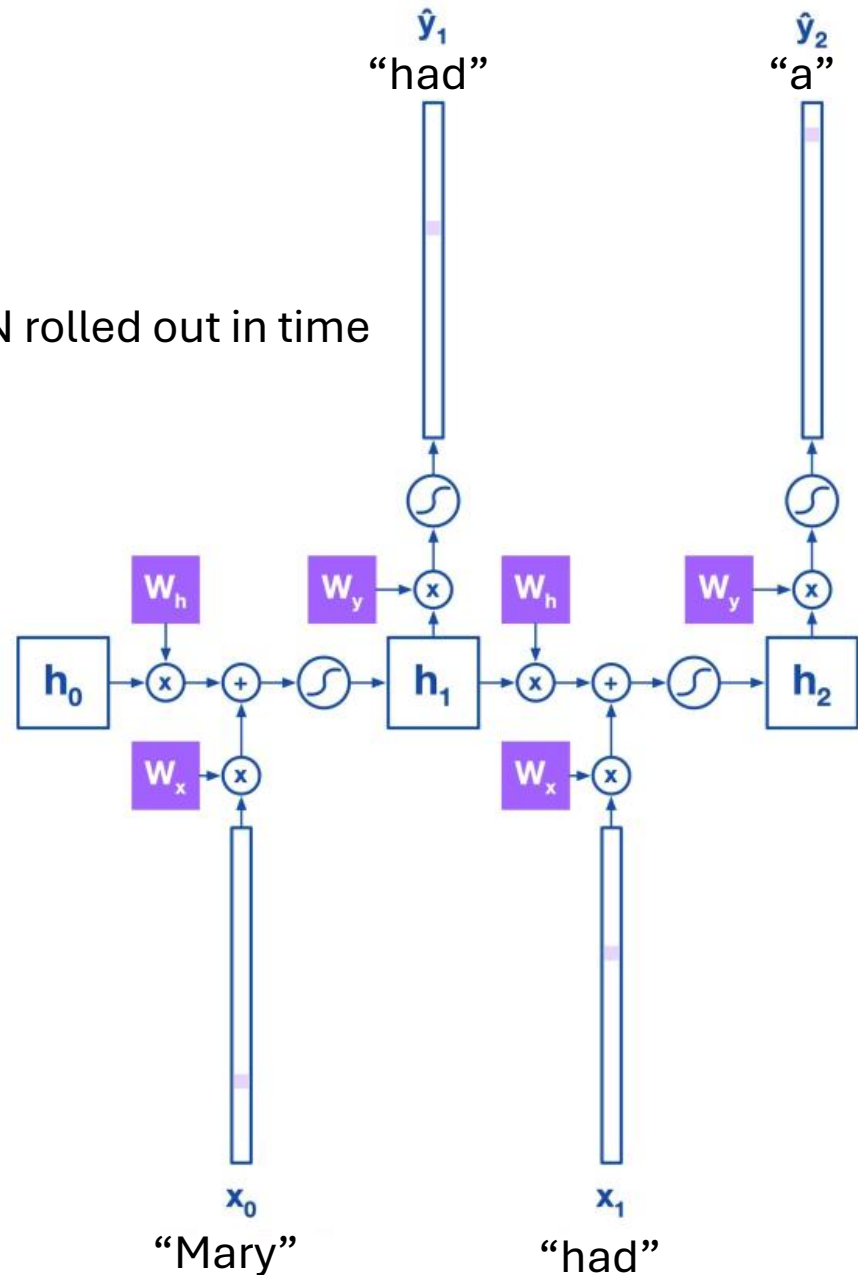
$$p(\hat{y}_{t+1}) = \text{softmax}(W_y h_t)$$

- Probability distribution over all English words

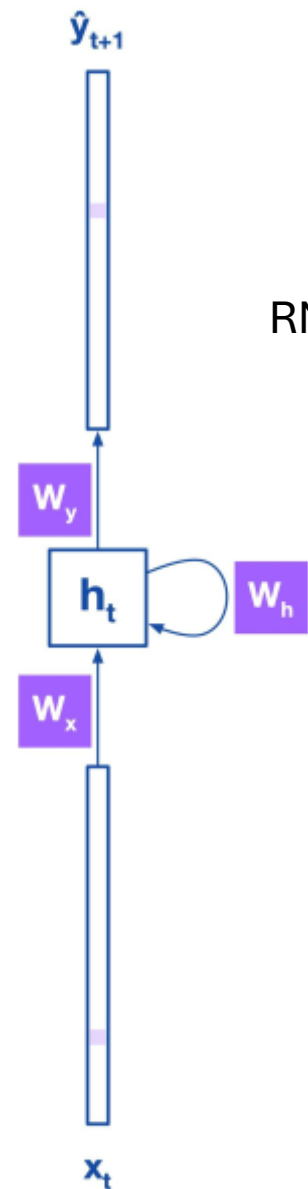


Source: Marta Garnelo, DeepMind

RNN rolled out in time



RNN structure

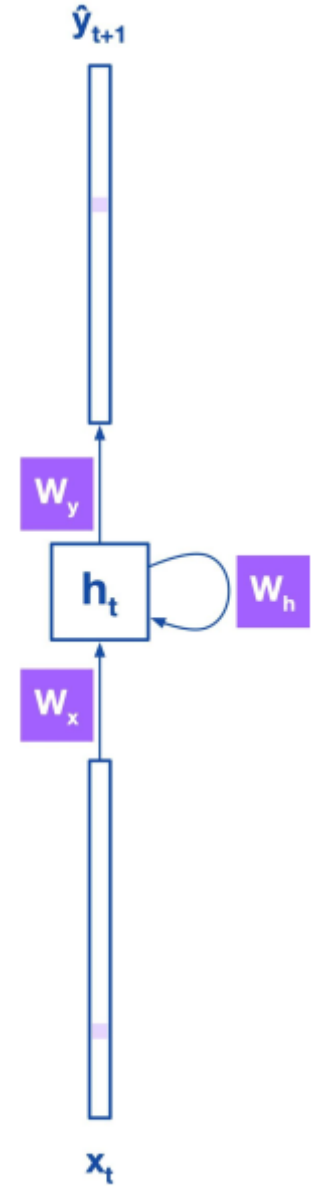


# Cross Entropy loss

- Essentially a classification task, where
  - number of classes = size of vocabulary

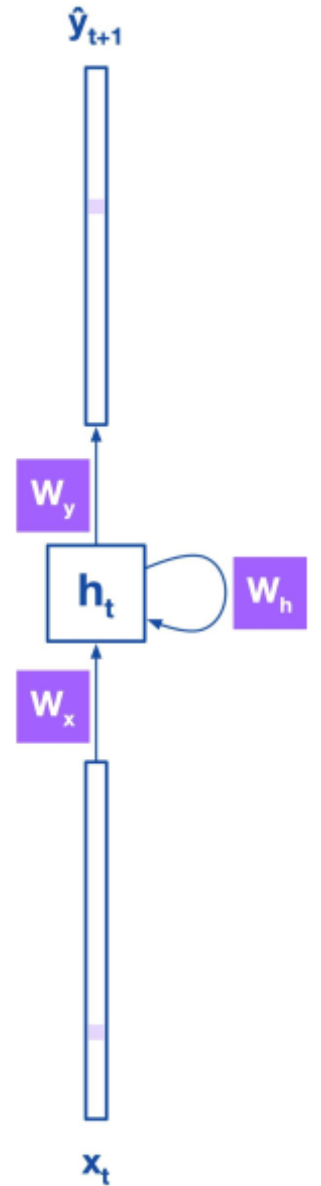
- $l(y_{i,t}, \hat{y}_t) = -y_{i,t} \log p(\hat{y}_t)$

$$\mathcal{L}(W_x, W_h, W_y) = \sum_{t=1}^T \sum_{i=1}^{N_{english}} l(y_{i,t}, \hat{y}_t)$$



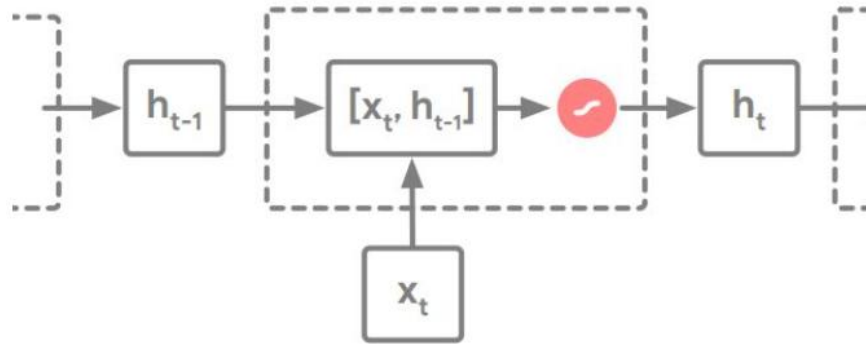
# Recurrent Neural Networks

- Desiderata:
  - ✓ • network must take **variable-length inputs**
  - ✓ •  $h$  must be **order-sensitive**
  - ✓ • **learnability** (differentiable: GD)
    - individual changes can have large effects (“not”)
    - preserve **long term dependencies**.

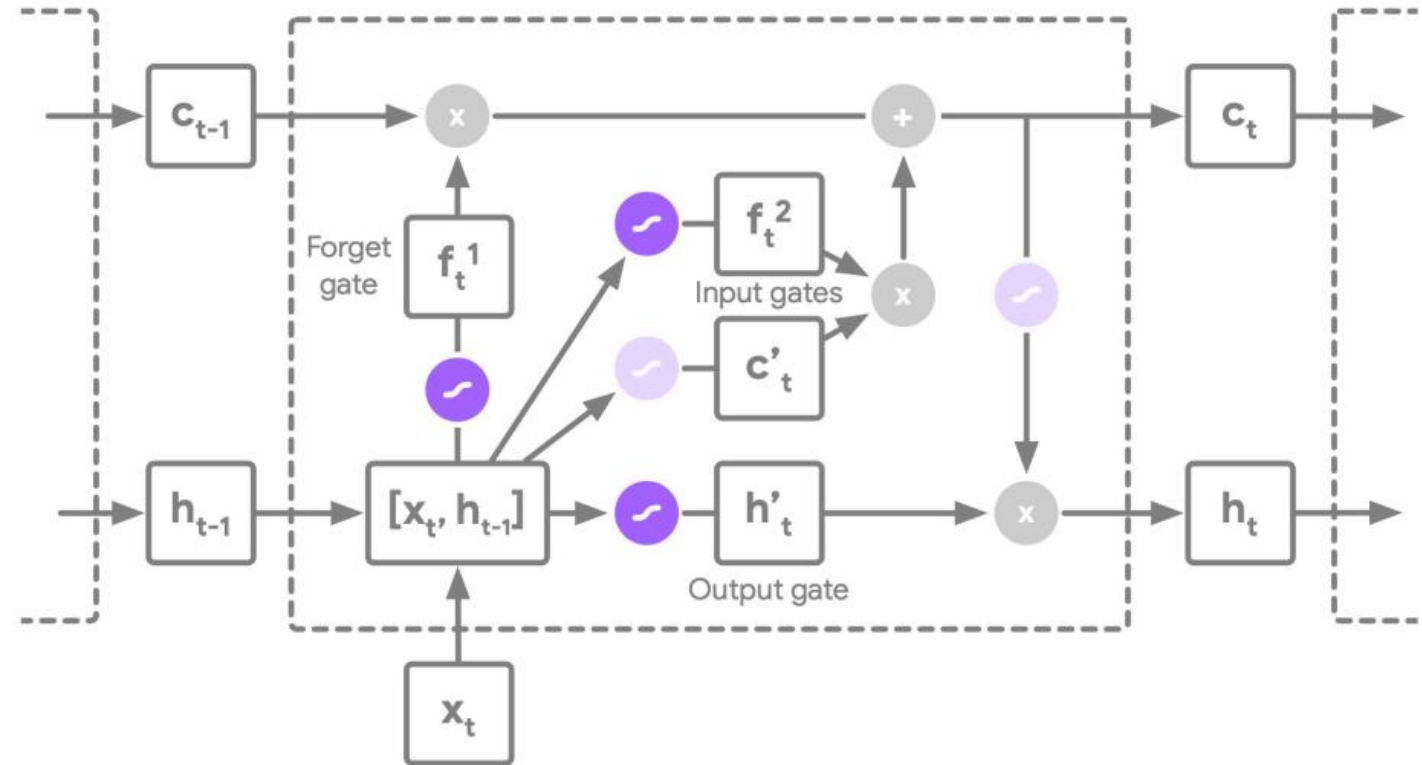


# Long Short-Term Memory

RNN state update

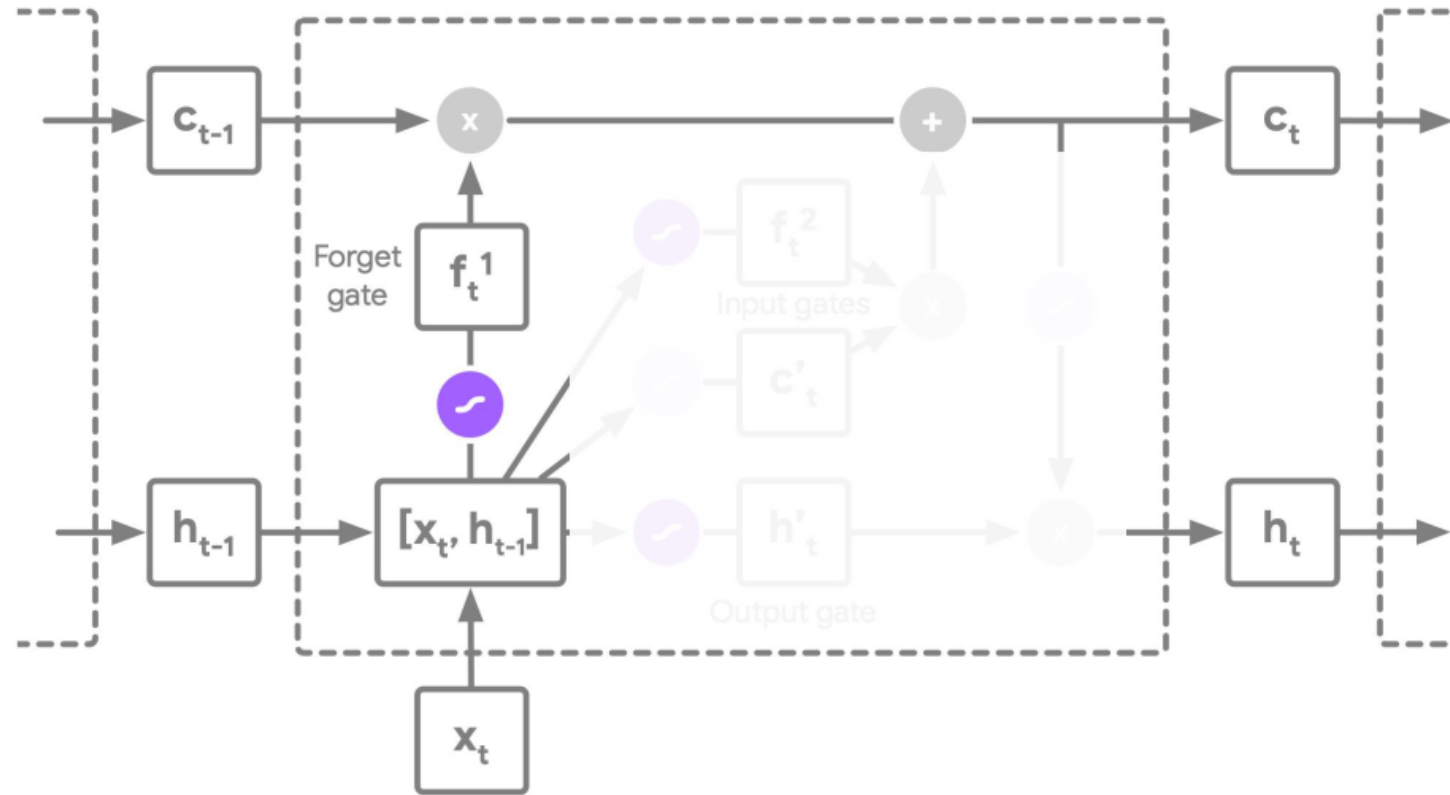


LSTM state update



# Long Short-Term Memory

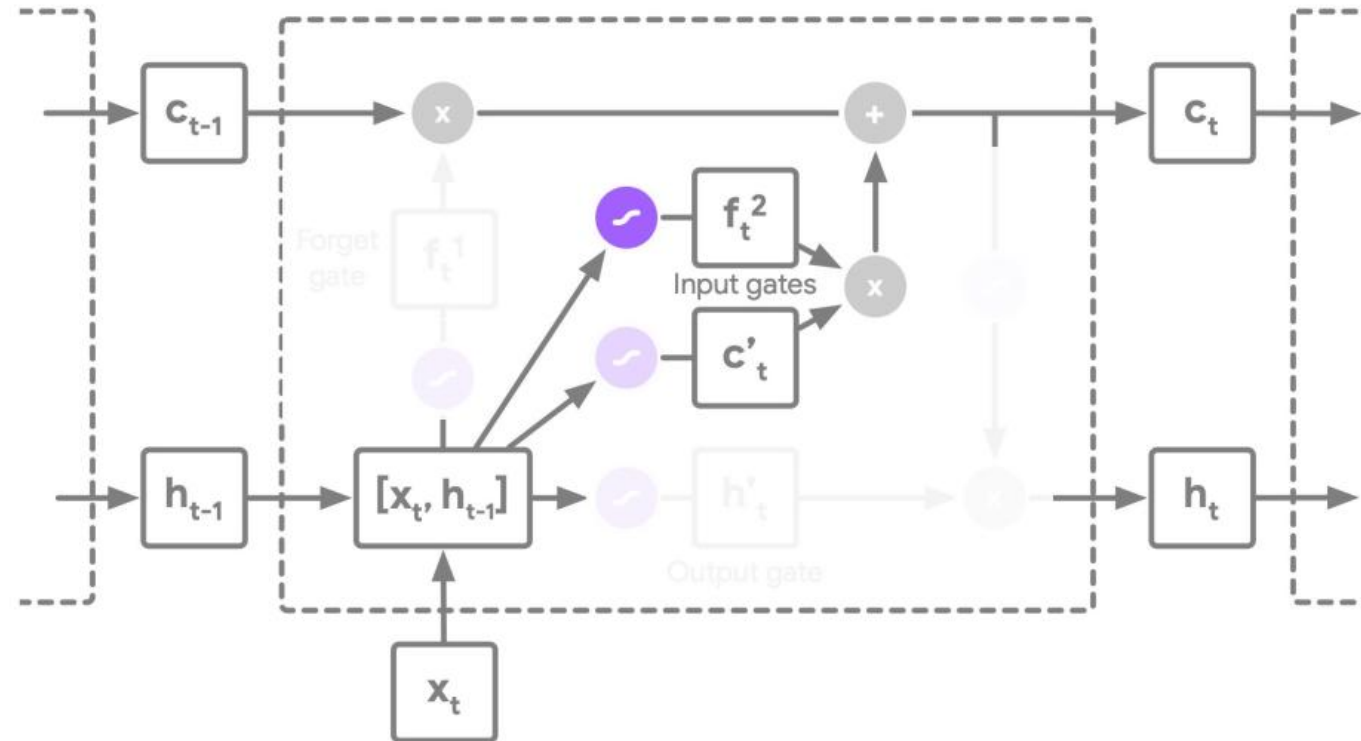
## 1) Forget gate



$$f_t^1 = \sigma(\mathbf{W}_{f^1} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_{f^1})$$

# Long Short-Term Memory

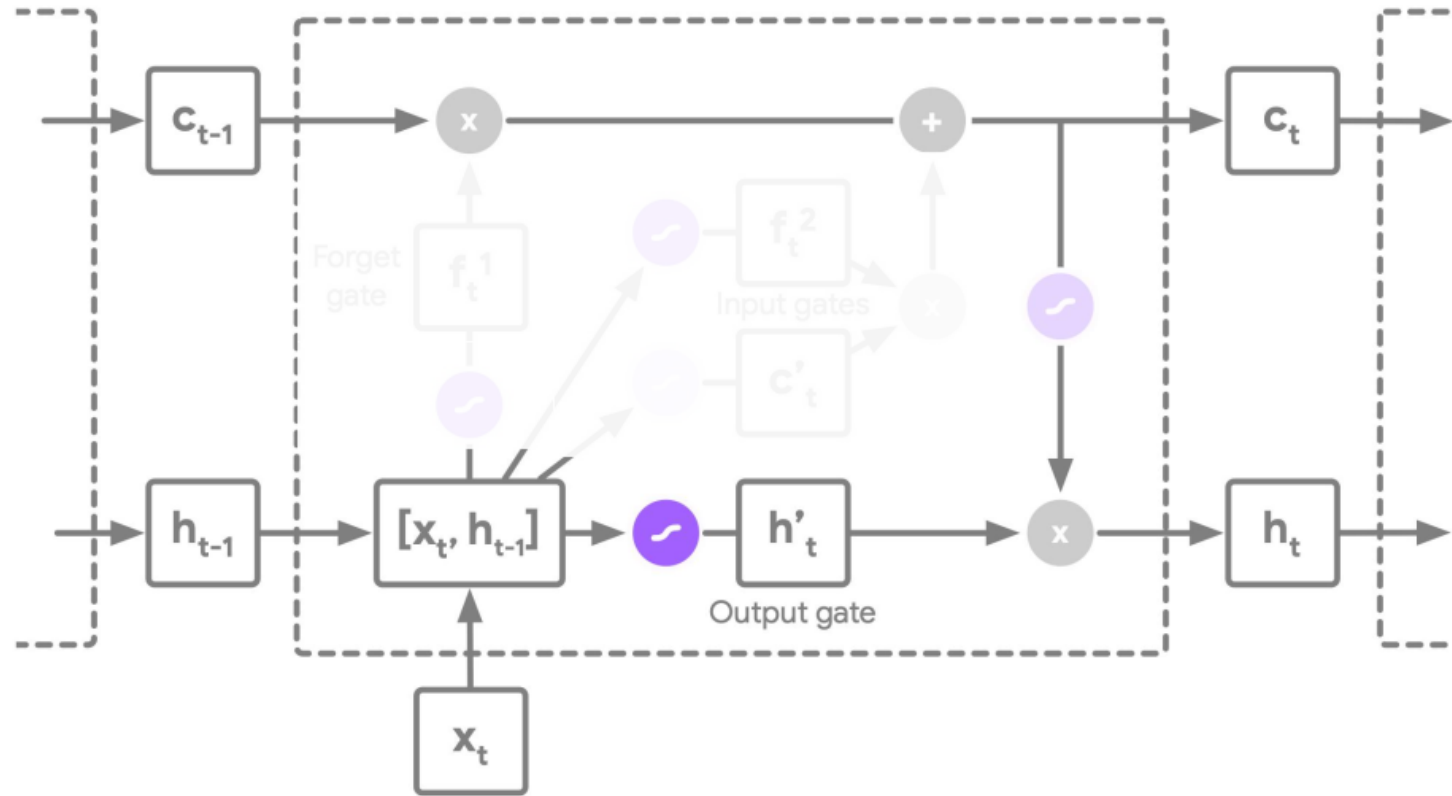
## 2) Input gates



$$f_t^2 = \sigma(\mathbf{W}_{f^2} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_{f^2}) \odot \tanh(\mathbf{W}_{f^2} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_{f^2})$$

# Long Short-Term Memory

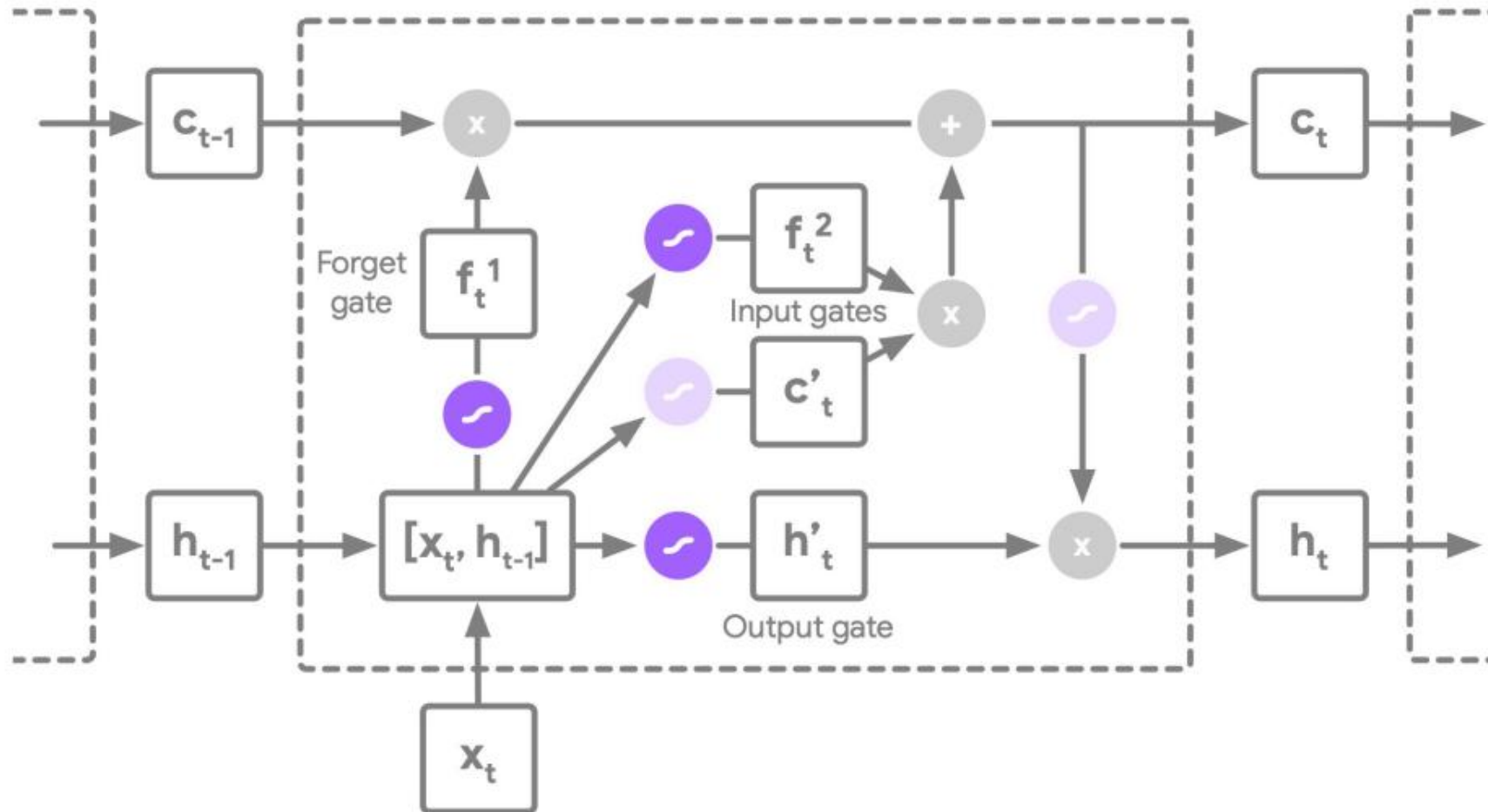
## 3) Output gate



$$h'_t = \sigma(\mathbf{W}_{h'_t} \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_{h'_t}) \odot \tanh(\mathbf{c}_t)$$

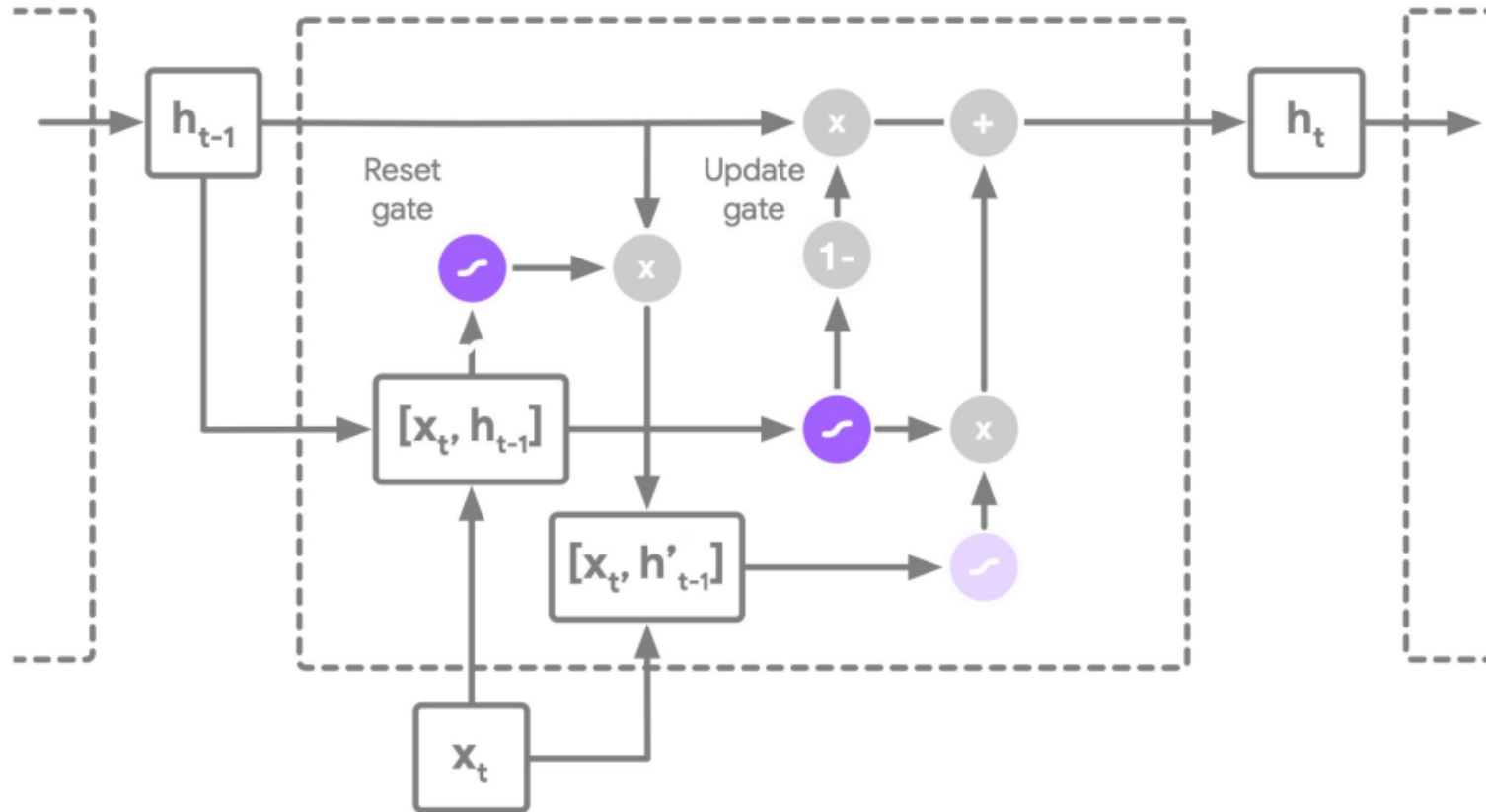
# Long Short-Term Memory

## LSTM state update



# Gated Recurrent Unit (GRU)

GRU state update



GRU can be seen as a simplified LSTM.